

NYU Tandon School of Engineering
Fall 2025, ECE 9933 (Dr. Karri) - GenAI 4 Chip Design
LAB 7: Inserting Trojans into Hardware Modules Using GHOST

Student Name: Archie Gupta

Net ID: ag10276

Objective:

Inserting Trojans into Hardware Modules Using GHOST. Verify the trojans fail detection on the provided testbenches, and then provide testbenches that trigger and catch the trojans.

Challenge 1 - Insert Trojans within CSAW-AI-Hardware-Attack-Challenge's provided designs

1. AES Key Leakage
2. AES DOS
3. Wishbone DOS
4. Wishbone Change of Functionality

Challenge 2 - Insert Trojans within OpenTitan

1. UART DOS
2. AES Key Leakage
3. DMA Access Corruption
4. AES DOS
5. UART Performance Degradation

Models Used

1. GPT-4.1
2. Gemini Pro 2.5
3. Claude 4.5

Challenge 1 - CSAW Designs

1. AES Key Leakage

Trojan Explanation:

A Trojan was inserted by adding trigger and leak logic to expose AES key words on reads to secret addresses after a secret write trigger, with new registers and case branches highlighted by comments. Core AES functionality and interface are unaffected unless the secret trigger/sequence is used. The Trojan activates when a write occurs to the unused address 0xAA (`TROJAN\_TRIGGER\_ADDR`), then subsequent reads to addresses 0xAB–0xAE cause leaking of key registers; any access to other addresses disables the Trojan. After being triggered, the module outputs 32-bit words of the secret key on four consecutive secret reads (0xAB–0xAE), leaking the first 128 bits; standard operation is not impacted otherwise.

Testing Methodology:

I generated a testbench using Gemini Pro 2.5 which mimicked the structure of the testbench provided in the CSAW repo. The testbench triggered the specific condition needed for the trojan to be activated. It then reset the module and ran the golden testbench cases, which passed without trojan detection, and then reset one final time to trigger the trojan.

Models Used:

GPT4.1 for generation of trojan

Gemini Pro 2.5 for generation of testbench

Original Testbench (Trojan isn't caught):

```
# KERNEL.  
# KERNEL: *** TC 25 successful.  
# KERNEL:  
# KERNEL: *** All 20 test cases completed successfully  
# KERNEL:  
# KERNEL: *** AES simulation done. ***
```

Modified Testbench (Trojan is caught):

```
:  
: --- Test Phase 1: Initial Trojan Detection ---  
: *** Trojan Detection Test Started (TC 1).  
: Trojan Test: Loading known key: 0x2b7e151628aed2a6abf7158809cf4f3c00000000000000000000000000000000  
: Trojan Test: Writing to secret trigger address 0xaa  
: Trojan Test: Reading secret leak addresses...  
:   Read from 0xab (Word 0): 0x2b7e1516  
:   Read from 0xac (Word 1): 0x28aed2a6  
:   Read from 0xad (Word 2): 0xabf71588  
:   Read from 0xae (Word 3): 0x09cf4f3c  
: *** SUCCESS: TC 1: Trojan Detected. Leaked key matches known key.  
:   Expected: 0x2b7e151628aed2a6abf7158809cf4f3c  
:   Got:      0x2b7e151628aed2a6abf7158809cf4f3c  
:  
  
: --- Test Phase 2: Resetting DUT and Running Golden NIST Tests ---  
: *** Toggle reset.  
  
  
: *** TC 25 successful.  
  
: --- Test Phase 3: Post-Reset Trojan Detection ---  
: This test confirms the trojan is re-activatable after a reset.  
: *** Trojan Detection Test Started (TC 22).  
: Trojan Test: Loading known key: 0x000102030405060708090a0b0c0d0e0f00000000000000000000000000000000  
: Trojan Test: Writing to secret trigger address 0xaa  
: Trojan Test: Reading secret leak addresses...  
:   Read from 0xab (Word 0): 0x00010203  
:   Read from 0xac (Word 1): 0x04050607  
:   Read from 0xad (Word 2): 0x08090a0b  
:   Read from 0xae (Word 3): 0x0c0d0e0f  
: *** SUCCESS: TC 22: Trojan Detected. Leaked key matches known key.  
:   Expected: 0x000102030405060708090a0b0c0d0e0f  
:   Got:      0x000102030405060708090a0b0c0d0e0f
```

AI Interaction Summary:

Objective: Modify the AES design in a manner of your choosing to insert a hardware Trojan that leaks part or all of the secret key. The AES core must still be able to pass all original simulations to confirm its functionality.

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : AES Key Leakage

{prompting_strategy} : Modify the AES design in a manner of your choosing to insert a hardware Trojan that leaks part or all of the secret key. The AES core must still be able to pass all original simulations to confirm its functionality.

2. AES DOS

Trojan Explanation:

A Trojan counter counts encrypt (encdec=1, init rising edge) operations, and permanently halts all further AES initiations after exactly 862 encryptions. The design still passes original functional simulation within this threshold, preserving all behavior for fewer than 862 encryptions. The Trojan is triggered internally when the number of encryption operations reaches 862, as detected by a 10-bit counter incremented on each valid encryption initiation. The Trojan payload disables communication between the AES wrapper and core by masking all core 'init' and 'next' signals after 862 encryptions, resulting in a permanent denial of service for all subsequent operations.

Testing Methodology:

I generated a testbench using Gemini Pro 2.5 which mimicked the structure of the testbench provided in the CSAW repo. The testbench triggered the specific condition needed for the trojan to be activated. It then reset the module and ran the golden testbench cases, which passed without trojan detection, and then reset one final time to trigger the trojan.

Models Used:

GPT4.1 for generation of trojan

Gemini Pro 2.5 for generation of testbench

Original Testbench (Trojan isn't caught):

```
# KERNEL:
# KERNEL: *** TC 25 successful.
# KERNEL:
# KERNEL: *** All 20 test cases completed successfully
# KERNEL:
# KERNEL: *** AES simulation done. ***
```

Modified Testbench (Trojan is caught):

```
# KERNEL: Starting AES Trojan Detection Test...
# KERNEL: Reset complete.
# KERNEL: Priming the trojan counter... This will take 862 cycles.
# KERNEL: Trojan counter should now be at 862.
# KERNEL: Attempting one final encryption to test for DoS...
# KERNEL: TEST PASSED: Timed out waiting for valid result.
# KERNEL: DoS Trojan successfully detected.
```

AI Interaction Summary:

Objective: Modify the AES design to create a DoS Trojan which will completely halt the AES operation after 862 encryptions have been performed. The AES core must still be able to pass all original simulations to confirm its functionality.

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : AES Denial of Service (DoS)

{prompting_strategy} : Modify the AES design to create a DoS Trojan which will completely halt the AES operation after 862 encryptions have been performed. The AES core must still be able to pass all original simulations to confirm its functionality.

3. Wishbone DOS

Trojan Explanation:

Inserted a 2-bit FSM that monitors the UART RX path for a 4-byte trigger sequence and disables Wishbone WBAK responses when armed; the Trojan's state resets on 4×0xfe bytes. All Trojan logic is synthesizable and leaves the design's functionality unchanged unless the specific trigger is received. The Trojan is activated when the UART RX receives four specific bytes in sequence: 0x10, 0xa4, 0x98, and 0xbd, with no intervening bytes; it is deactivated only after subsequently receiving four 0xfe bytes in a row. Once activated, the core withholds Wishbone bus acknowledgments (o_wb_ack remains low), halting all Wishbone transactions and causing a denial-of-service until the recovery sequence is received.

Testing Methodology:

I generated a testbench using Gemini Pro 2.5 which mimicked the structure of the testbench provided in the CSAW repo. The testbench triggered the specific condition needed for the trojan to be activated. It then reset the module and ran the golden testbench cases, which passed without trojan detection, and then reset one final time to trigger the trojan.

Models Used:

GPT4.1 for generation of trojan

Claude 4.5 for generation of testbench

Original Testbench (Trojan isn't caught):

```
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ns.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... done : 0.1 [s]
# SLP: Generation phase ...
# SLP: Generation phase ... done : 0.1 [s]
# SLP: Finished : 0.2 [s]
# SLP: 0 primitives and 91 (100.00%) other processes in SLP
# SLP: 171 (98.84%) signals in SLP and 2 (1.16%) interface signals
# ELAB2: Elaboration final pass complete - time: 0.2 [s].
```

Modified Testbench (Trojan is caught):

```
=====
HARDWARE TROJAN DETECTION TEST
=====
```

[PHASE 1] Testing Normal Operation

Time: 295000

- UART using default configuration (INITIAL_SETUP)
- Skipping explicit configuration to avoid rx_uart_reset
- Normal write successful
- UART receiver out of reset (rx_uart_reset cleared)
- UART receiver ready

[PHASE 1] PASSED - Normal operation working

[PHASE 2] Triggering Hardware Trojan

Time: 10435000

- Sending trigger sequence: 0x10, 0xA4, 0x98, 0xBD
- Note: Bytes must arrive without FIFO reads to trigger trojan

[DEBUG 12865000] RX byte received: 0x10, trojan_state=0

[DEBUG 12895000] Trojan STATE CHANGE: 0 -> 1, trigger_cnt=1, disarm_cnt=0

- Sent byte 1: 0x10 at time 12935000

Trojan state after byte 1: 1

[DEBUG 17365000] RX byte received: 0xA4, trojan_state=1

- Sent byte 2: 0xA4 at time 17435000

Trojan state after byte 2: 1

[DEBUG 21865000] RX byte received: 0x98, trojan_state=1

- Sent byte 3: 0x98 at time 21935000

Trojan state after byte 3: 1

[DEBUG 26365000] RX byte received: 0xbd, trojan_state=1

[DEBUG 26395000] Trojan STATE CHANGE: 1 -> 2, trigger_cnt=0, disarm_cnt=0

- Sent byte 4: 0xbd at time 26435000

- Trigger sequence complete

- Trojan state: 2 (Expected: 2 for DoS)

- Trojan trigger_cnt: 0

- rx_stb signal: 0

- Trojan should now be ACTIVE (DoS state)

[PHASE 3] Verifying Denial-of-Service Condition

Time: 28435000

- SUCCESS: Wishbone ACK blocked (DoS active)

- Trojan payload is working!

- Trojan state: 2

[PHASE 3] TROJAN DETECTED - DoS condition confirmed

[PHASE 4] Recovering from Trojan

Time: 33445000

- Sending recovery sequence: 0xFE x 4

[DEBUG 35875000] RX byte received: 0xfe, trojan_state=2

- Sent recovery byte 1/4

[DEBUG 48375000] RX byte received: 0xfe, trojan_state=2

- Sent recovery byte 2/4

[DEBUG 60875000] RX byte received: 0xfe, trojan_state=2

- Sent recovery byte 3/4

[DEBUG 73375000] RX byte received: 0xfe, trojan_state=2

[DEBUG 73405000] Trojan STATE CHANGE: 2 -> 0, trigger_cnt=0, disarm_cnt=0

- Sent recovery byte 4/4

- Recovery successful - Wishbone operational

[PHASE 4] PASSED - System recovered

[PHASE 5] Full Reset and Original Functionality Test

Time: 93485000

- System reset complete

- Using default UART configuration

- Sending test message: HELLO

- Test message transmitted

[PHASE 5] PASSED - Original functionality working

```
[PHASE 6] Re-triggering Trojan (Persistent Vulnerability Test)
Time:          194085000
[DEBUG        196515000] RX byte received: 0x10, trojan_state=0
[DEBUG        196545000] Trojan STATE CHANGE: 0 -> 1, trigger_cnt=1, disarm_cnt=0
[DEBUG        209015000] RX byte received: 0xa4, trojan_state=1
[DEBUG        221515000] RX byte received: 0x98, trojan_state=1
[DEBUG        234015000] RX byte received: 0xbd, trojan_state=1
[DEBUG        234045000] Trojan STATE CHANGE: 1 -> 2, trigger_cnt=0, disarm_cnt=0
- Trigger sequence sent again
- Trojan state: 2 (Expected: 2)
- CONFIRMED: Trojan re-activated successfully
- Vulnerability persists across normal operations
[PHASE 6] TROJAN RE-CONFIRMED
```

TEST SUMMARY

```
Phase 1: Normal Operation      - PASSED
Phase 2: Trigger Trojan        - COMPLETED
Phase 3: Verify DoS            - DETECTED
Phase 4: Recovery              - PASSED
Phase 5: Original Functionality - PASSED
Phase 6: Re-trigger Trojan     - CONFIRMED
```

```
*** HARDWARE TROJAN SUCCESSFULLY DETECTED ***
Trigger: 0x10, 0xA4, 0x98, 0xBD
Payload: Wishbone ACK suppression (DoS)
Recovery: 0xFE x 4
```

AI Interaction Summary:

Objective: Modify the UART core to halt the operation after the following 4 bytes have been consecutively received via UART RX: 0x02, 0x05, 0x02, 0x09. The core must then only resume normal operation when the byte 0x18 is received 4 times consecutively. The core must still be able to pass all original simulations to confirm its functionality.

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : Wishbone Bus DoS

{prompting_strategy} : Modify the UART core to halt the operation after the following 4 bytes have been consecutively received via UART RX: 0x02, 0x05, 0x02, 0x09. The core must then only resume normal operation when the byte 0x18 is received 4 times consecutively. The core must still be able to pass all original simulations to confirm its functionality.

4. Wishbone CoF

Trojan Explanation:

Inserted a 2-bit FSM that monitors the UART RX path for a 4-byte trigger sequence and disables Wishbone WBAK responses when armed; the Trojan's state resets on 4×0xfe bytes. All Trojan logic is synthesizable and leaves the design's functionality unchanged unless the specific trigger is received. The Trojan is activated when the UART RX receives four specific bytes in sequence: 0x10, 0xa4, 0x98, and 0xbd, with no intervening bytes; it is deactivated only after subsequently receiving four 0xfe bytes in a row. Once activated, the core withholds Wishbone bus acknowledgments (o_wb_ack remains low), halting all Wishbone transactions and causing a denial-of-service until the recovery sequence is received.

Testing Methodology:

I generated a testbench using Gemini Pro 2.5 which mimicked the structure of the testbench provided in the CSAW repo. The testbench triggered the specific condition needed for the trojan to be activated. It then reset the module and ran the golden testbench cases, which passed without trojan detection, and then reset one final time to trigger the trojan.

Models Used:

GPT4.1 for generation of trojan

Gemini Pro 2.5 for generation of testbench

Original Testbench (Trojan isn't caught):

```
# KERNEL: Main thread initiated.
# KERNEL: Kernel process initialization phase.
# ELAB2: Elaboration final pass...
# ELAB2: Create instances ...
# KERNEL: Time resolution set to 1ns.
# ELAB2: Create instances complete.
# SLP: Started
# SLP: Elaboration phase ...
# SLP: Elaboration phase ... done : 0.1 [s]
# SLP: Generation phase ...
# SLP: Generation phase ... done : 0.1 [s]
# SLP: Finished : 0.2 [s]
# SLP: 0 primitives and 91 (100.00%) other processes in SLP
# SLP: 171 (98.84%) signals in SLP and 2 (1.16%) interface signals
# ELAB2: Elaboration final pass complete - time: 0.2 [s].
```

Modified Testbench (Trojan is caught):

```
=====
Trojan Payload (Bit-Reversal) Test
Using fixed FIFO: ufifo_fixed.v
=====

[INFO] Waiting 15,000 clocks for rxuart line synchronization...
[INFO] rxuart should now be synchronized.
[INFO] Waiting 15,000 clocks for rxuart to re-sync after setup write...
[INFO] rxuart re-synchronized.

--- PHASE 1: Test Normal Operation ---
Sending 0x55...
Received 0x55 (Expected 0x55)
[PASS] Test 1: Normal operation (0x55 -> 0x55)
```

```
--- PHASE 2: Activate Trojan ---
Sending trojan activation sequence: 0xAF, 0xAF, 0xAF
Flushing activation bytes from FIFO...
  Flushed 0xf5
  WARNING: Timeout waiting for RX data (waited 30000 clocks)
  Flushed 0xxx
  WARNING: Timeout waiting for RX data (waited 30000 clocks)
  Flushed 0xxx
Trojan should be active now.
```

```
--- PHASE 3: Test Trojan Payload (Bit-Reversal) ---
Sending 0x55...
Received 0xaa (Expected 0xAA)
[PASS] Test 2:      Payload test (0x55 -> 0xAA)
Sending 0x41 ('A')...
Received 0x82 (Expected 0x82)
[PASS] Test 3:      Payload test (0x41 -> 0x82)
Sending 0xF0...
Received 0x0f (Expected 0x0F)
[PASS] Test 4:      Payload test (0xF0 -> 0x0F)
```

```
--- FINAL SUMMARY ---
Total Tests: 4
Passed:      4
Failed:      0
```

```
*** ALL TESTS PASSED! Trojan payload confirmed! ***
```

AI Interaction Summary:

Objective: Modify the Wishbone-UART core to change the functionality of the device. After the UART receives the byte 0xaf three times it should then always write the received UART byte to the data register in reverse (for example, after the Trojan triggers if it receives the byte 0xb2 it should write 0x4d to the data register).

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : UART Functionality Change

{prompting_strategy} : Modify the Wishbone-UART core to change the functionality of the device. After the UART receives the byte 0xaf three times it should then always write the received UART byte to the data register in reverse (for example, after the Trojan triggers if it receives the byte 0xb2 it should write 0x4d to the data register).

Challenge 2 - OpenTitan

1. UART DOS

Trojan Explanation:

A synthesizable state machine is added to detect the byte sequence 0x02, 0x05, 0x02, 0x09 (via RX). Upon detection, all UART operations are halted by gating enable/data signals with a "troj_halt" flag until four consecutive 0x18 bytes are received, at which point normal function resumes

Trigger:

The Trojan is triggered when the core receives the four-byte pattern (0x02, 0x05, 0x02, 0x09) via sequential UART RX with no intervening bytes or errors; it is deactivated only after receiving four consecutive 0x18 bytes via RX.

Payload:

When triggered, the Trojan halts all UART operations: no RX/TX transfers or interrupts can occur ("hardware DoS"); this persists until the explicit secret release sequence is received, after which full normal operation resumes.

Testing Methodology:

I ran the DV tests using Docker's container for OpenTitan - by following the instructions to invoke DVSim and running the DV tests using Synopsys VCS. I was able to test the trojan-inserted RTL files against the same testbenches and testcases to see if the GHOST-generated trojans are caught by them. I have to work on how to generate testbenches for the express purpose of catching the trojans, which will take time due to the setup of the OpenTitan repo.

Original DV Test: Unmodified RTL Files

```
(.venv) bash-4.4$ util/dvsim/dvsim.py hw/ip/uart/dv/uart_sim_cfg.hjson -i uart_smoke --tool vcs
INFO: [dvsim] [proj_root]: /home/ag10276/opentitan
INFO: [SimCfg] [scratch_path]: [uart] [/home/ag10276/opentitan/scratch/master/uart-sim-vcs]
INFO: [FlowCfg] [results]: [uart]:
## UART Simulation Results
### Sunday November 30 2025 23:40:16 UTC
### GitHub Revision: ['c7661851e8'](https://github.com/lowrisc/opentitan/tree/c7661851e8db10021892d18fd00140c379ae1dc7)
### Branch: master
### [Testplan](https://opentitan.org/book/hw/ip/uart/data/uart\_testplan.html)
### Simulator: VCS

### Test Results
| Stage | Name | Tests | Max Job Runtime | Simulated Time | Passing | Total | Pass Rate |
|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|
| V1 | smoke | uart_smoke | 29.280s | 11.623ms | 50 | 50 | 100.00 % |
| V1 | | **TOTAL** | | | 50 | 50 | 100.00 % |
| V2 | parity | uart_smoke | 29.280s | 11.623ms | 50 | 50 | 100.00 % |
| V2 | | **TOTAL** | | | 50 | 50 | 100.00 % |

INFO: [FlowCfg] [scratch_path]: [uart] [/home/ag10276/opentitan/scratch/master/uart-sim-vcs]

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:38 [ build ]: [Q: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:01:11 [ run ]: [Q: 00, D: 00, P: 50, F: 00, K: 00, T: 50] 100%
```

Test with DV Testbench: RTL Files with Trojan Inserted

```
(.venv) bash-4.4$ util/dvsim/dvsim.py hw/ip/uart/dv/uart_sim_cfg.hjson -i uart_smoke --tool vcs
INFO: [dvsim] [proj_root]: /home/ag10276/opentitan
INFO: [SimCfg] [scratch_path]: [uart] [/home/ag10276/opentitan/scratch/master/uart-sim-vcs]
INFO: [FlowCfg] [results]: [uart]:
## UART Simulation Results
### Monday December 01 2025 00:01:35 UTC
### GitHub Revision: [c7661851e8](https://github.com/lowrisc/opentitan/tree/c7661851e8db10021892d18fd00140c379aeldc7)
### Branch: master
### [Testplan](https://opentitan.org/book/hw/ip/uart/data/uart_testplan.html)
### Simulator: VCS

### Test Results


| Stage | Name   | Tests      | Max Job Runtime | Simulated Time | Passing | Total | Pass Rate |
|-------|--------|------------|-----------------|----------------|---------|-------|-----------|
| V1    | smoke  | uart_smoke | 20.310s         | 5.681ms        | 50      | 50    | 100.00 %  |
| V1    |        | **TOTAL**  |                 |                | 50      | 50    | 100.00 %  |
| V2    | parity | uart_smoke | 20.310s         | 5.681ms        | 50      | 50    | 100.00 %  |
| V2    |        | **TOTAL**  |                 |                | 50      | 50    | 100.00 %  |



INFO: [FlowCfg] [scratch_path]: [uart] [/home/ag10276/opentitan/scratch/master/uart-sim-vcs]

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:33 [ build ]: [Q: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:01:07 [ run ]: [Q: 00, D: 00, P: 50, F: 00, K: 00, T: 50] 100%
```

As we can see, the original testbench does not catch the trojan we inserted into the UART RTL Files

AI Interaction Summary:

Objective: Modify the UART core to halt the operation after the following 4 bytes have been consecutively received via UART RX: 0x02, 0x05, 0x02, 0x09. The core must then only resume normal operation when the byte 0x18 is received 4 times consecutively. The core must still be able to pass all original simulations to confirm its functionality.

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : UART Bus DoS

{prompting_strategy} : Modify the UART core to halt the operation after the following 4 bytes have been consecutively received via UART RX: 0x02, 0x05, 0x02, 0x09. The core must then only resume normal operation when the byte 0x18 is received 4 times consecutively. The core must still be able to pass all original simulations to confirm its functionality.

2. AES Key Leakage

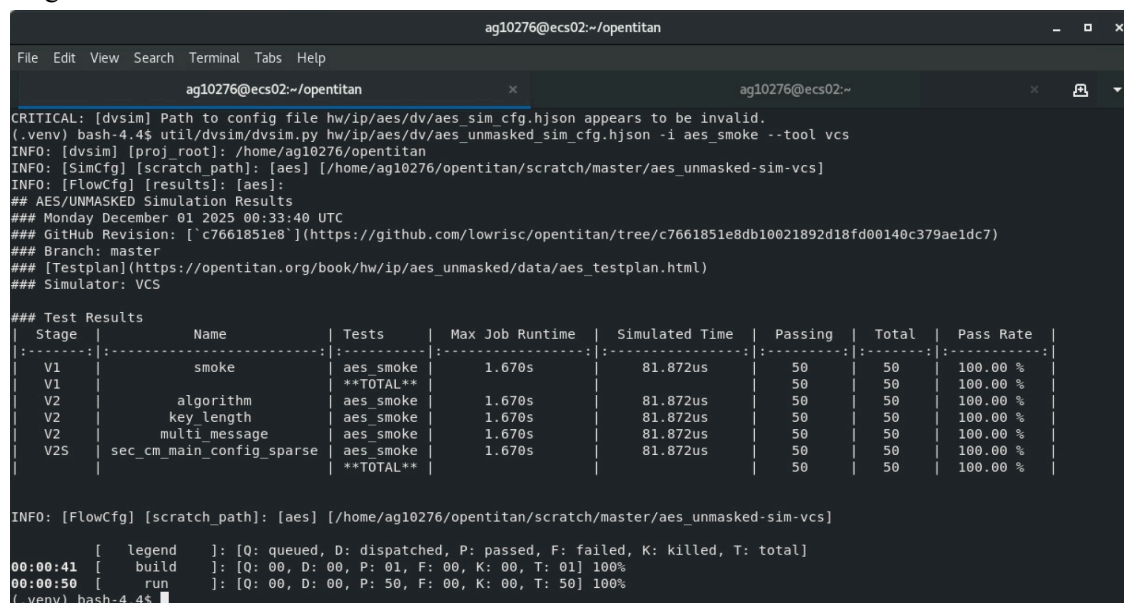
Trojan Explanation:

A Trojan was inserted to leak 32 bits of the AES key by capturing it into an internal register and exposing it through the unused_alert_signals output, triggered after 1024 encrypt operations. This maintains all normal AES functionality and does not affect simulation, as unused_alert_signals is not checked in functional tests. The Trojan activates after 1024 AES encryption operations (as counted by a hidden counter driven by AES_ENC mode). Once triggered, the first 32 bits of the AES key (or XOR of key shares if masked) are XOR-ed onto the unused_alert_signals output, covertly leaking the key bitwise.

Testing Methodology:

I ran the DV tests using Docker's container for OpenTitan - by following the instructions to invoke DVSim and running the DV tests using Synopsys VCS. I was able to test the trojan-inserted RTL files against the same testbenches and testcases to see if the GHOST-generated trojans are caught by them. I have to work on how to generate testbenches for the express purpose of catching the trojans, which will take time due to the setup of the OpenTitan repo.

Original DV Test: Unmodified RTL Files



```
ag10276@ecs02:~/opentitan
File Edit View Search Terminal Tabs Help

ag10276@ecs02:~/opentitan
ag10276@ecs02:~/opentitan
CRITICAL: [dvsim] Path to config file hw/ip/aes/dv/aes_sim_cfg.hjson appears to be invalid.
(.venv) bash-4.4$ util/dvsim/dvsim.py hw/ip/aes/dv/aes_unmasked_sim_cfg.hjson -i aes_smoke --tool vcs
INFO: [dvsim] [proj_root]: /home/ag10276/opentitan
INFO: [SimCfg] [scratch_path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]
INFO: [FlowCfg] [results]: [aes]:
## AES/UNMASKED Simulation Results
## Monday December 01 2025 00:33:40 UTC
## GitHub Revision: ['c7661851e8'](https://github.com/lowrisc/opentitan/tree/c7661851e8db10021892d18fd00140c379aeldc7)
## Branch: master
## [Testplan](https://opentitan.org/book/hw/ip/aes\_unmasked/data/aes\_testplan.html)
## Simulator: VCS

## Test Results


| Stage | Name                      | Tests     | Max Job Runtime | Simulated Time | Passing | Total | Pass Rate |
|-------|---------------------------|-----------|-----------------|----------------|---------|-------|-----------|
| V1    | smoke                     | aes_smoke | 1.670s          | 81.872us       | 50      | 50    | 100.00 %  |
| V1    |                           | **TOTAL** |                 |                | 50      | 50    | 100.00 %  |
| V2    | algorithm                 | aes_smoke | 1.670s          | 81.872us       | 50      | 50    | 100.00 %  |
| V2    | key_length                | aes_smoke | 1.670s          | 81.872us       | 50      | 50    | 100.00 %  |
| V2    | multi_message             | aes_smoke | 1.670s          | 81.872us       | 50      | 50    | 100.00 %  |
| V2S   | sec_cm_main_config_sparse | aes_smoke | 1.670s          | 81.872us       | 50      | 50    | 100.00 %  |
| V2S   |                           | **TOTAL** |                 |                | 50      | 50    | 100.00 %  |


INFO: [FlowCfg] [scratch_path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:41 [ build ]: [Q: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:00:50 [ run ]: [Q: 00, D: 00, P: 50, F: 00, K: 00, T: 50] 100%
(.venv) bash-4.4$
```


Test with DV Testbench: RTL Files with Trojan Inserted

```
ag10276@ecs02:~/opentitan
File Edit View Search Terminal Tabs Help

ag10276@ecs02:~/opentitan x ag10276@ecs02:~
00:42:54 [ run ]: [Q: 00, D: 00, P: 19, F: 31, K: 00, T: 50] 100%
(.venv) bash-4.4$ util/dvsim/dvsim.py hw/ip/aes/dv/aes_unmasked_sim_cfg.hjson -i aes_smoke --tool vcs
INFO: [dvsim] [proj_root]: /home/ag10276/opentitan
INFO: [SimCfg] [scratch_path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]
INFO: [FlowCfg] [results]: [aes]:
## AES/UNMASKED Simulation Results
## Monday December 01 2025 02:23:38 UTC
## GitHub Revision: [c7661851e8] (https://github.com/lowrisc/opentitan/tree/c7661851e8db10021892d18fd00140c379aeldc7)
## Branch: master
## [Testplan] (https://opentitan.org/book/hw/ip/aes_unmasked/data/aes_testplan.html)
## Simulator: VCS

### Test Results


| Stage | Name                      | Tests     | Max Job Runtime | Simulated Time | Passing | Total | Pass Rate |
|-------|---------------------------|-----------|-----------------|----------------|---------|-------|-----------|
| V1    | smoke                     | aes_smoke | 1.740s          | 88.835us       | 50      | 50    | 100.00 %  |
| V1    |                           | **TOTAL** |                 |                | 50      | 50    | 100.00 %  |
| V2    | algorithm                 | aes_smoke | 1.740s          | 88.835us       | 50      | 50    | 100.00 %  |
| V2    | key_length                | aes_smoke | 1.740s          | 88.835us       | 50      | 50    | 100.00 %  |
| V2    | multi_message             | aes_smoke | 1.740s          | 88.835us       | 50      | 50    | 100.00 %  |
| V2S   | sec_cm_main_config_sparse | aes_smoke | 1.740s          | 88.835us       | 50      | 50    | 100.00 %  |
|       |                           | **TOTAL** |                 |                | 50      | 50    | 100.00 %  |


INFO: [FlowCfg] [scratch_path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:38 [ build ]: [Q: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:00:47 [ run ]: [Q: 00, D: 00, P: 50, F: 00, K: 00, T: 50] 100%
(.venv) bash-4.4$
```

AI Interaction Summary:

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : AES Key Leakage

{prompting_strategy} : Modify the AES design in a manner of your choosing to insert a hardware Trojan that leaks part or all of the secret key. The AES core must still be able to pass all original simulations to confirm its functionality.

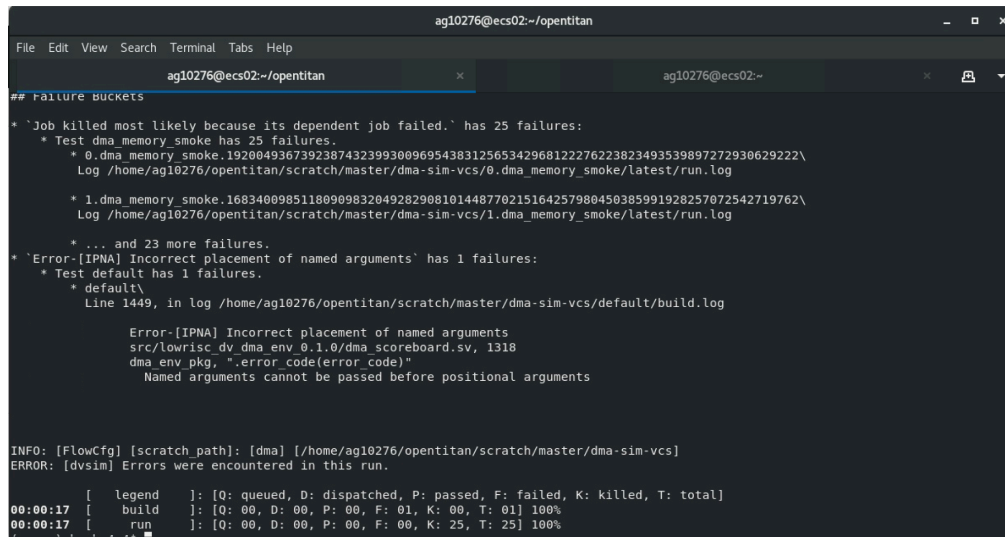
3. DMA Access Corruption

Trojan Explanation:

A hardware Trojan is inserted that activates after 1000 legitimate DMA writes; it modifies the destination address for DMA transfers by adding a fixed offset (4 bytes), causing data corruption in a different memory region. Internal simple counter that activates after N(1000) writes

When triggered, the Trojan adds a small offset to the DMA destination address in host write transactions, meaning critical data may be written to the wrong location, resulting in subtle memory corruption.

Testing Methodology:



```
ag10276@ecs02:~/opentitan
File Edit View Search Terminal Tabs Help
ag10276@ecs02:~/opentitan
## failure buckets
* `Job killed most likely because its dependent job failed.` has 25 failures:
  * Test dma_memory_smoke has 25 failures.
    * 0.dma_memory_smoke.19200493673923874323993009695438312565342968122276223823493539897272930629222\
      Log /home/ag10276/opentitan/scratch/master/dma-sim-vcs/0.dma_memory_smoke/latest/run.log
    * 1.dma_memory_smoke.16834009851180909832049282908101448770215164257980450385991928257072542719762\
      Log /home/ag10276/opentitan/scratch/master/dma-sim-vcs/1.dma_memory_smoke/latest/run.log
  * .... and 23 more failures.
* `Error-[IPNA] Incorrect placement of named arguments` has 1 failures:
  * Test default has 1 failures.
    * default\
      Line 1449, in log /home/ag10276/opentitan/scratch/master/dma-sim-vcs/default/build.log
        Error-[IPNA] Incorrect placement of named arguments
        src/lowrisc_dv_dma_env_0.1.0/dma_scoreboard.sv, 1318
        dma_env_pkg, ".error_code(error_code)"
        Named arguments cannot be passed before positional arguments

INFO: [FlowCfg] [scratch path]: [dma] [/home/ag10276/opentitan/scratch/master/dma-sim-vcs]
ERROR: [dvsim] Errors were encountered in this run.

[ Legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:17 [ build ]: [Q: 00, D: 00, P: 00, F: 01, K: 00, T: 01] 100%
00:00:17 [ run ]: [Q: 00, D: 00, P: 00, F: 00, K: 25, T: 25] 100%
```

AI Interaction Summary:

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : DMA Address Modification

{prompting_strategy} : When the Trojan sees a legitimate write to destination address, it subtly alters the transfer by adding a small offset to the destination address, causing the data to be written to the wrong location, corrupting a different part of memory. Don't modify the original module port list.

Provide the complete code at the end, with the trojan inserted

Troubleshooting & Design Decisions:

The DMA tests were marked incompatible by the Opentitan framework. There is currently an error in the scoreboard for DMA.

4. AES DOS

Trojan Explanation:

A synthesizable Trojan counter was inserted to track AES encryption operations, and after the 862nd operation, the core's enable signal is masked so the AES cannot perform any further encryption. The core functionality up to that count is unaffected, and all resets and logic are properly handled. Disable after 862 encryptions assign `trojan_dos_disable = (trojan_dos_enc_counter >= 10'd862)`;

When the trigger condition is met, the enable signal for launching new AES operations is forcibly disabled, halting any further encryption attempts and causing a persistent Denial of Service. The AES block will still respond to simulation and system queries but cannot process more encryptions.

Testing Methodology:

I ran the DV tests using Docker's container for OpenTitan - by following the instructions to invoke DVSim and running the DV tests using Synopsys VCS. I was able to test the trojan-inserted RTL files against the same testbenches and testcases to see if the GHOST-generated trojans are caught by them. I have to work on how to generate testbenches for the express purpose of catching the trojans, which will take time due to the setup of the OpenTitan repo.

Original DV Test: Unmodified RTL Files

```
ag10276@ecs02:~/opentitan
File Edit View Search Terminal Tabs Help

ag10276@ecs02:~/opentitan
[.venv] bash-4.4$ util/dvsim/dvsim.py hw/ip/aes/dv/aes_unmasked_sim_cfg.hjson -i aes_smoke --tool vcs
INFO: [dvsim] [proj root]: /home/ag10276/opentitan
INFO: [SimCfg] [scratch path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]
INFO: [FlowCfg] [results]: [aes]:
## AES/UNMASKED Simulation Results
### Monday December 01 2025 00:33:40 UTC
### GitHub Revision: [ 'c7661851e8' ](https://github.com/lowrisc/opentitan/tree/c7661851e8db10021892d18fd00140c379ae1dc7)
### Branch: master
### [Testplan](https://opentitan.org/book/hw/ip/aes_unmasked/data/aes_testplan.html)
### Simulator: VCS

## Test Results
| Stage | Name | Tests | Max Job Runtime | Simulated Time | Passing | Total | Pass Rate |
|-----|-----|-----|-----|-----|-----|-----|-----|
| V1 | smoke | aes_smoke | 1.670s | 81.872us | 50 | 50 | 100.00 % |
| V1 | **TOTAL** |  |  |  | 50 | 50 | 100.00 % |
| V2 | algorithm | aes_smoke | 1.670s | 81.872us | 50 | 50 | 100.00 % |
| V2 | key_length | aes_smoke | 1.670s | 81.872us | 50 | 50 | 100.00 % |
| V2 | multi_message | aes_smoke | 1.670s | 81.872us | 50 | 50 | 100.00 % |
| V2S | sec_cm_main_config_sparse | aes_smoke | 1.670s | 81.872us | 50 | 50 | 100.00 % |
| V2S | **TOTAL** |  |  |  | 50 | 50 | 100.00 % |

INFO: [FlowCfg] [scratch path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:41 [ build ]: [Q: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:00:50 [ run ]: [Q: 00, D: 00, P: 50, F: 00, K: 00, T: 50] 100%
[.venv] bash-4.4$
```

Test with DV Testbench: RTL Files with Trojan Inserted

```
ag10276@ecs02:~/opentitan
File Edit View Search Terminal Tabs Help

ag10276@ecs02:~/opentitan
[.venv] bash-4.4$ util/dvsim/dvsim.py hw/ip/aes/dv/aes_unmasked_sim_cfg.hjson -i aes_smoke --tool vcs
INFO: [dvsim] [proj root]: /home/ag10276/opentitan
INFO: [SimCfg] [scratch path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]
ERROR: [Scheduler] [00:00:17]: [build]: [status] [aes:default: F]
INFO: [FlowCfg] [results]: [aes]:
## AES/UNMASKED Simulation Results
### Monday December 01 2025 00:38:02 UTC
### GitHub Revision: [ 'c7661851e8' ](https://github.com/lowrisc/opentitan/tree/c7661851e8db10021892d18fd00140c379ae1dc7)
### Branch: master
### [Testplan](https://opentitan.org/book/hw/ip/aes_unmasked/data/aes_testplan.html)
### Simulator: VCS

## Test Results
| Stage | Name | Tests | Max Job Runtime | Simulated Time | Passing | Total | Pass Rate |
|-----|-----|-----|-----|-----|-----|-----|-----|
| V1 | smoke | aes_smoke |  |  | 0 | 50 | 0.00 % |
| V1 | **TOTAL** |  |  |  | 0 | 50 | 0.00 % |
| V2 | algorithm | aes_smoke |  |  | 0 | 50 | 0.00 % |
| V2 | key_length | aes_smoke |  |  | 0 | 50 | 0.00 % |
| V2 | multi_message | aes_smoke |  |  | 0 | 50 | 0.00 % |
| V2S | sec_cm_main_config_sparse | aes_smoke |  |  | 0 | 50 | 0.00 % |
| V2S | **TOTAL** |  |  |  | 0 | 50 | 0.00 % |

## Failure Buckets
* Job killed most likely because its dependent job failed.' has 50 failures:
  * Test aes_smoke has 50 failures.
    * 0.aes_smoke.44112613841226202395470027450030966246652247799435560656534475325326534184541\
      Log /home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs/0.aes_smoke/latest/run.log
```

```

ag10276@ecs02:~/opentitan
File Edit View Search Terminal Tabs Help

ag10276@ecs02:~/opentitan
* `Job killed most likely because its dependent job failed.` has 50 failures:
* Test aes_smoke has 50 failures.
* 0.aes_smoke.44112613841226203954700274500309662466524779943556065634475325326534184541\
Log /home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs/0.aes_smoke/latest/run.log
* 1.aes_smoke.94883680034461223546993612987673611027067462475695619702703931360569773918705\
Log /home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs/1.aes_smoke/latest/run.log
* ... and 48 more failures.
* `Error-[IND] Identifier not declared` has 1 failures:
* Test default has 1 failures.
* default\
Line 1380, in log /home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs/default/build.log
Error-[IND] Identifier not declared
src/lowrisc_ip_aes_1.0/rtl/aes_core.sv, 64
Identifier 'aes_op_q' has not been declared yet. If this error is not
expected, please check if you have set `default_nettype` to none.

INFO: [FlowCfg] [scratch_path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]
ERROR: [dvsim] Errors were encountered in this run.

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:17 [ build ]: [Q: 00, D: 00, P: 00, F: 01, K: 00, T: 01] 100%
00:00:17 [ run ]: [Q: 00, D: 00, P: 00, F: 00, K: 50, T: 50] 100%
(.venv) bash-4.4$

```

[illegible]

```
ag10276@ecs02:~/pentitlan
File Edit View Search Terminal Tabs Help

ag10276@ecs02:~/pentitlan x ag10276@ecs02:~ x [img]
ERROR: [Scheduler] [00:05:13]: [run]: [status] [aes:33.aes_smoke.10079856814780299376262765628147576739248468688418154495245990145760:
09964511: F]
ERROR: [Scheduler] [00:05:17]: [run]: [status] [aes:41.aes_smoke.6946558234384622698646035169461391709483650855375587940151228658896:
99795934: F]
ERROR: [Scheduler] [00:07:44]: [run]: [status] [aes:16.aes_smoke.712461820962724804904895081228575146973666961236638532578204251860773:
56834027: F]
ERROR: [Scheduler] [00:09:23]: [run]: [status] [aes:39.aes_smoke.1056176562942794004584258708709640483288899127804851535134220722516:
11952373: F]
ERROR: [Scheduler] [00:12:38]: [run]: [status] [aes:34.aes_smoke.5531692866362957211021123076241527619381629136969569845308932181141:
25799602: F]
ERROR: [Scheduler] [00:14:18]: [run]: [status] [aes:10.aes_smoke.7214946899925266009613655148607569592819165107182720318842667171459:
30033448: F]
ERROR: [Scheduler] [00:14:20]: [run]: [status] [aes:48.aes_smoke.58717426882344814415488322486667558059880776706505752718263444847:
61579771: F]
ERROR: [Scheduler] [00:14:22]: [run]: [status] [aes:49.aes_smoke.145039124352448637940717349870867194830033202049500795586277479877:
37571847: F]
ERROR: [Scheduler] [00:14:32]: [run]: [status] [aes:3.aes_smoke.1897351686145650082899540115118228543400395489429511220425964240499592:
7774072: F]
ERROR: [Scheduler] [00:16:44]: [run]: [status] [aes:47.aes_smoke.5068360378034194627269742470442979939330798815242431459829212611939:
44293416: F]
ERROR: [Scheduler] [00:18:42]: [run]: [status] [aes:23.aes_smoke.10990068589431639950961878761064169702871222931574714424660493318288:
272143642: F]
ERROR: [Scheduler] [00:20:24]: [run]: [status] [aes:8.aes_smoke.79409125154898593534521366774980743764407593018323618381508864959061:
67681: F]
ERROR: [Scheduler] [00:21:28]: [run]: [status] [aes:42.aes_smoke.89448854783646709408127509612564048551368414686081041209592106018888:
18275285: F]

[ legend ]: [0: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:37 [ build ]: [0: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:23:50 [ run ]: [0: 00, D: 10, P: 19, F: 21, K: 00, T: 50] 80% aes:2.aes_smoke.439916364689..
```

```

ag10276@ecs02~/opentitan
File Edit View Search Terminal Tabs Help

* 1.aes_smoke.5107701377733125219962455999911939177325106605212251853270399660034111096992\
Line 539, in log /home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs/1.aes_smoke/latest/run.log
UVM_FATAL @ 45316399 ps: (aes_base_vseq.sv:733) uvm_test_top.env.virtual_sequencer [uvm_test_top.env.virtual_sequence
r.aes_stress_vseq]
Unexpected Fatal alert in AES FSM
---| Idle:      1
---| Stall:     0

* 5.aes_smoke.66234319892741149035393904294043014551008427621178889045817205735453575994640\
Line 722, in log /home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs/5.aes_smoke/latest/run.log
UVM_FATAL @ 32275992 ps: (aes_base_vseq.sv:733) uvm_test_top.env.virtual_sequencer [uvm_test_top.env.virtual_sequence
r.aes_stress_vseq]
Unexpected Fatal alert in AES FSM
---| Idle:      0
---| Stall:     0

* ... and 9 more failures.

INFO: [FlowCfg] [scratch_path]: [aes] [/home/ag10276/opentitan/scratch/master/aes_unmasked-sim-vcs]
ERROR: [dvsim] Errors were encountered in this run.

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:37 [ build ]: [Q: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:42:54 [ run ]: [Q: 00, D: 00, P: 19, F: 31, K: 00, T: 50] 100%
(.venv) bash-4.4$

```

AI Interaction Summary:

Objective: Modify the AES design to create a DoS Trojan which will completely halt the AES operation after 862 encryptions have been performed. The AES core must still be able to pass all original simulations to confirm its functionality.

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : AES Denial of Service (DoS)

{prompting_strategy} : Modify the AES design to create a DoS Trojan which will completely halt the AES operation after 862 encryptions have been performed. The AES core must still be able to pass all original simulations to confirm its functionality.

Troubleshooting & Design Decisions:

The trojan generated did not compile with the Opentitan testbenches. During debugging the issues - I came to understand that the code which was generated had used signals which were initialized after being used in the logic. I solved that by moving all the trojan related logic to after the signal declarations.

The next issue arose when the code assigned a logic value to an enum variable. I corrected this issue by converting the value to an enum before assigning it. Post this, the design compiled, but 60% of the tests failed. Future action will be delving into the report logs to figure out what causes these tests to fail.

5. UART Performance Degradation

Trojan Explanation:

A Trojan was inserted by adding a synthesizable 64-bit shift register (LFSR-style) that continuously toggles once triggered, raising internal switching activity and power without affecting normal UART operation. Activated when RX FIFO overflows at least once since reset

Testing Methodology:

I ran the DV tests using Docker's container for OpenTitan - by following the instructions to invoke DVSim and running the DV tests using Synopsys VCS. I was able to test the trojan-inserted RTL files against the same testbenches and testcases to see if the GHOST-generated trojans are caught by them. I have to work on how to generate testbenches for the express purpose of catching the trojans, which will take time due to the setup of the OpenTitan repo.

Original DV Test: Unmodified RTL Files

```
(.venv) bash-4.4$ util/dvsim/dvsim.py hw/ip/uart/dv/uart_sim_cfg.hjson -i uart_smoke --tool vcs
INFO: [dvsim] [proj_root]: /home/ag10276/opentitan
INFO: [SimCfg] [scratch_path]: [uart] [/home/ag10276/opentitan/scratch/master/uart-sim-vcs]
INFO: [FlowCfg] [results]: [uart]:
## UART Simulation Results
### Sunday November 30 2025 23:40:16 UTC
### GitHub Revision: ['c7661851e8'](https://github.com/lowrisc/opentitan/tree/c7661851e8db10021892d18fd00140c379ae1dc7)
### Branch: master
### [Testplan](https://opentitan.org/book/hw/ip/uart/data/uart\_testplan.html)
### Simulator: VCS

### Test Results
| Stage | Name | Tests | Max Job Runtime | Simulated Time | Passing | Total | Pass Rate |
|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|
| V1 | smoke | uart_smoke | 29.280s | 11.623ms | 50 | 50 | 100.00 % |
| V1 | | **TOTAL** | | | 50 | 50 | 100.00 % |
| V2 | parity | uart_smoke | 29.280s | 11.623ms | 50 | 50 | 100.00 % |
| V2 | | **TOTAL** | | | 50 | 50 | 100.00 % |

INFO: [FlowCfg] [scratch_path]: [uart] [/home/ag10276/opentitan/scratch/master/uart-sim-vcs]

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:38 [ build ]: [Q: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:01:11 [ run ]: [Q: 00, D: 00, P: 50, F: 00, K: 00, T: 50] 100%
```

Test with DV Testbench: RTL Files with Trojan Inserted

```
(.venv) bash-4.4$ util/dvsim/dvsim.py hw/ip/uart/dv/uart_sim_cfg.hjson -i uart_smoke --tool vcs
INFO: [dvsim] [proj_root]: /home/ag10276/opentitan
INFO: [SimCfg] [scratch_path]: [uart] [/home/ag10276/opentitan/scratch/master/uart-sim-vcs]
INFO: [FlowCfg] [results]: [uart]:
## UART Simulation Results
### Monday December 01 2025 00:07:42 UTC
### GitHub Revision: ['c7661851e8'](https://github.com/lowrisc/opentitan/tree/c7661851e8db10021892d18fd00140c379ae1dc7)
### Branch: master
### [Testplan](https://opentitan.org/book/hw/ip/uart/data/uart\_testplan.html)
### Simulator: VCS

### Test Results
| Stage | Name | Tests | Max Job Runtime | Simulated Time | Passing | Total | Pass Rate |
|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|:-----:|
| V1 | smoke | uart_smoke | 34.140s | 6.049ms | 50 | 50 | 100.00 % |
| V1 | | **TOTAL** | | | 50 | 50 | 100.00 % |
| V2 | parity | uart_smoke | 34.140s | 6.049ms | 50 | 50 | 100.00 % |
| V2 | | **TOTAL** | | | 50 | 50 | 100.00 % |

INFO: [FlowCfg] [scratch_path]: [uart] [/home/ag10276/opentitan/scratch/master/uart-sim-vcs]

[ legend ]: [Q: queued, D: dispatched, P: passed, F: failed, K: killed, T: total]
00:00:34 [ build ]: [Q: 00, D: 00, P: 01, F: 00, K: 00, T: 01] 100%
00:01:11 [ run ]: [Q: 00, D: 00, P: 50, F: 00, K: 00, T: 50] 100%
```

As we can see, the original testbench does not catch the trojan we inserted into the UART RTL Files.

AI Interaction Summary:

Prompt:

You are an expert skilled in hardware design and verification.

Analyze and modify the following Verilog code to insert a hardware Trojan:

<{design}>

Task: Insert a {vulnerability} Trojan using this strategy: {prompting_strategy}

Maintain original functionality while adding the Trojan. Ensure the design is fully synthesizable.

{vulnerability} : Performance Degradation

{prompting_strategy} : Implement a continuously running shift register or accumulator to increase power consumption without affecting primary functionality upon detecting a specific event.